The Safety Net With a Hole: Coverage Drift Between Failure States and Recovery States in a Self-Healing Workflow Runtime

The Safety Net With a Hole: Coverage Drift Between Failure States and Recovery States in a Self-Healing Workflow Runtime

Research companion to the Facetwork thesis (`thesis.md`). Empirical basis: the July 2026 continuation-liveness investigation — a disproven fix, a read-only forensic redo, and a verified low-risk fix; all commits, probes, and workflow-state records are in the Facetwork repository and its fleet's execution history.

Abstract

Leaderless workflow runtimes lean on *safety-net* recovery loops — periodic sweeps, reapers, watchdogs — that scan persisted state for work that has stalled and nudge it forward. Such a loop encodes, explicitly or not, a set of **states that need rescue**; correctness requires that set to equal the set of **states at which work can actually strand**. Nothing enforces the equality, and it drifts as the state machine grows. We report a production liveness bug that was exactly this drift: a workflow step that entered error-recovery (`CATCH_BEGIN`) could strand indefinitely because the catch states had never been added to the sweep's rescue set, even as they became genuinely-blocking states. The bug had defied diagnosis for months, manifesting as deep fan-out workflows that froze at their fan-in tail and required manual "re-seeding" to finish. We describe two things of independent interest: (1) a *disproven* first fix that reasoned from code to a plausible cause (optimistic-concurrency sequence regression) and, by changing write semantics on that hypothesis, converted a deadlock into a **livelock** (13,870 self-triggering recovery tasks in one run) before being reverted; and (2) the disciplined redo — a read-only forensic probe that froze a live stall, localized the frontier to a single catch-stranded step, and *confirmed* the mechanism by seeding one recovery task and watching the step advance in six seconds. The real fix is a read-query change with no write-semantics risk: a single canonical set of stall-states referenced by every sweep site. Verification on the workload that had never once completed without babysitting: it completed hands-off, zero re-seeds, identical output. The transferable lessons are a named failure mode (*coverage drift*) with a structural remedy (one authoritative stall-state set), and a methodological rule the false start earned the hard way: **for liveness bugs, observe the frozen system before you mutate a write path.**

1. Introduction

A distributed workflow step in Facetwork advances through a persisted state machine, and *any* runner may perform the next transition by claiming a lightweight *continuation* task. Normally, completing one step generates continuations for the parents it unblocks, and the cascade walks itself to completion. When that cascade drops a link — a continuation that should have been created but wasn't, or a transition that never got triggered — a step can sit non-terminal with nothing pointing at it. The engine has a safety net for exactly this: a periodic **stuck-step sweep** that scans for steps in known-blocking states and resumes them.

For months, deep fan-out workflows (a continental "emergency-access atlas", tens of thousands of steps) stalled at their fan-in tail: a plateau of non-terminal steps, zero in-flight tasks, zero pending continuations, the runner reporting "running," no progress. The operational workaround was a *reseed driver* — a babysitting loop that manually re-enqueued continuations for every non-terminal step, advancing the workflow one generation per push. It worked, but it meant no deep run ever finished unattended, and the sweep — the very mechanism designed to prevent this — visibly failed to help.

This paper is the account of finding out why. It has an unusual shape because the investigation did: the first fix was *wrong in an instructive way*, and the second succeeded only by abandoning reasoning-from-code for reading-from-the-running-system. We contribute:

1. **A named failure mode** (§6): *coverage drift* between the set of states at which work strands and the set a safety-net loop rescues — a silent, growing liveness hole in any self-healing recovery system.
2. **A cautionary case** (§4): a plausible, code-reasoned fix that changed write semantics and made the failure strictly worse (deadlock → livelock), with the mechanism of the amplification.
3. **A read-only forensic method** (§5) that localized and *confirmed* the real cause without perturbing the system, and the six-second experiment that settled it.
4. **A low-risk structural fix and its measured effect** (§7): one authoritative stall-state set; before/after on the same workload.

2. Background

Continuations and the cascade. Each step lives in the database. When a step reaches a terminal state, the iteration that completed it generates continuation tasks (`_fw_continue`) for the parent blocks it unblocks; a runner claims each and performs the next transition. This is the normal engine of progress — no sweep involved.

The stuck-step sweep. Because the cascade can drop links (a commit race, a crash between generating a transition and enqueuing its continuation, a runner dying mid-iteration), each runner periodically runs a sweep:

1. *Select* workflows that have any step in a blocking state (`get_pending_resume_workflow_ids`).
2. For each, *fetch* the stuck steps (`get_stuck_steps_for_workflow`) and resume them (an idempotent, deterministic re-derivation).

Both queries filter on a hardcoded set of states. The sweep runs on a jittered ~5-minute cadence and is deliberately rate-limited.

The catch clause. Error recovery (`catch`) routes a failed step into a recovery sub-machine: the step transitions to `CATCH_BEGIN`, a recovery sub-block is created (`CATCH_CONTINUE` observes it to completion), and its yield supplies the step's results. Catch was a comparatively recent feature; it introduced two new genuinely-blocking states, `CATCH_BEGIN` and `CATCH_CONTINUE`, at which a step waits for the recovery sub-machine to be created and to finish. *(That these states themselves took a multi-round campaign to make work distributed is a separate paper; here they are simply new blocking states in the machine.)*

3. The symptom, precisely

The stall signature: a workflow with N non-terminal steps, **zero** pending or running tasks (including zero `_fw_continue`), runner state "running," no step advancing for many minutes. It was scale-sensitive — small runs eventually crawled to completion, large ones did not within any patient timeout — which is the fingerprint of a recovery mechanism that is *present but too slow or too partial* rather than absent.

Two facts framed the search. First, the sweep exists and covers block-level blocking states, so a wholesale "the sweep doesn't run" theory was wrong — it ran, and workflows *were* being selected by it. Second, re-seeding continuations by hand always worked, which proved the stuck steps were *processable* — they were not deadlocked on missing data or a genuine dependency, merely un-triggered. The question was why the cascade dropped the trigger and why the sweep did not supply it.

4. The false start: a write-path fix that made it worse

Reasoning from the code, a plausible culprit presented itself. Steps carry an optimistic-concurrency *sequence* for their persisted version; the batch commit advances a step only if the database copy is strictly behind (compare-and-set). Fleet logs showed recurring CAS conflicts on the *same* step IDs at sweep cadence — "our sequence=1, DB already at >= it" — dropping writes. And a direct-save code path wrote the caller's in-memory sequence verbatim, which could *regress* the counter. The hypothesis wrote itself: a stale save resets the CAS baseline, racing writers fight to re-advance from zero, and legitimate transitions get dropped. The observed CAS-conflict warnings were real, so the hypothesis felt confirmed.

The fix followed the hypothesis: make the sequence server-authoritative (an atomic `$inc` on save, never trusting the caller), and — since a dropped transition otherwise strands its parents — enqueue a *self-wake continuation* when a batch write lost the CAS, so "someone re-derives from the winning state."

It made the bug dramatically worse. On the verification run the workflow was *more* stuck, not less, and the database held **13,870** self-wake continuations for one run. Two amplifying mechanisms, both consequences of touching write semantics:

- **Dual sequence authority.** The batch-commit path still managed the sequence in its CAS; the new `$inc` managed it independently on every direct save. Two writers advancing the same counter by different rules produced *more* CAS conflicts, not fewer — the exact metric the fix was meant to reduce.
- **A self-triggering recovery loop.** Each CAS conflict now enqueued a continuation; claiming it re-derived and re-attempted the write; which conflicted again against the still-racing counter; which enqueued another. A deadlock (nothing progressing) had become a **livelock** (a great deal progressing, none of it forward). The de-duplication guard (one pending wake per step) capped the *instantaneous* count but not the *cumulative* churn, because each wake completed before the next conflict created its successor.

We reverted the entire change. The lesson, which §5 then acted on: **the CAS conflicts were a real signal, but the hypothesis mistook a symptom for the mechanism, and validating it required changing write semantics — the one category of change that can convert a stall into a storm.** A liveness bug in a concurrent system is precisely where reasoning-from-code is least trustworthy, because the failure lives in interleavings the code does not locally reveal.

5. Read-only forensics

The redo obeyed a single rule: *change nothing until the frozen system has been read.* We built a read-only probe that, given a stalled workflow, reports its structure without mutation:

- Every non-terminal step by state.
- The **frontier**: non-terminal steps all of whose children are terminal — the deepest stuck points, the ones that *should* be advancing.
- For each frontier step, whether any task or continuation targets it.
- A **dry run**: seed a throwaway in-memory store with the workflow's steps and ask the evaluator to process a frontier step there, observing whether it would advance — against a copy, so the real database is untouched.

To catch a stall before the slow sweep erased it, we triggered a modest run (six regions, one designed to fail so the catch path fired) and polled at twelve-second resolution, freezing on the plateau. The probe's output was unambiguous:

```
non-terminal: 6  (2 blocks.Continue, 3 execution.Continue, 1 catch.Begin)
in-flight tasks: 1 (a lingering root task); pending _fw_continue: 0
frontier: 1 step, state = catch.Begin, seq 0, age 154s,
          4 children (all terminal), live tasks targeting it: NONE
```

The frontier was a single step at `catch.Begin` — entered catch recovery, never processed, no continuation, no task, stuck two and a half minutes and counting. The five other non-terminal steps were its ancestors, at block-continue states, each correctly waiting on it.

Rather than trust the dry run alone (its AST-loading path hit an unrelated snag), we ran the definitive experiment, and it is the crux of the paper. We seeded **one** continuation task pointed at the frozen step — adding a task, not changing any write path — and watched:

```
BEFORE: catch.Begin, seq 0
+6s:    ADVANCED -> catch.Continue
```

Six seconds. The step was fully processable the entire time; it had simply never been triggered, and nothing in the safety net was going to trigger it. That single observation converted the investigation from hypothesis to fact: the stall is not a data problem, not a race that corrupts state, not the CAS issue the first fix chased — it is a step in a blocking state that *no mechanism is looking for*.

6. The root cause: coverage drift

With the frontier state known, the cause was a two-line read of the code. The sweep's queries filter on a hardcoded state set:

```
{ EVENT_TRANSMIT, STATEMENT_BLOCKS_BEGIN, STATEMENT_BLOCKS_CONTINUE,
  BLOCK_EXECUTION_BEGIN, BLOCK_EXECUTION_CONTINUE }
```

`CATCH_BEGIN` and `CATCH_CONTINUE` are absent. They are genuinely-blocking states — a step waits at them for the recovery sub-machine — but they were added to the state machine (with the catch feature) and *never added here*. The safety net had a hole precisely the shape of error recovery.

The scale-sensitivity and the "sweep runs but doesn't help" paradox both fall out of this. The stalled workflow *was* selected for sweeping — because its block-continue ancestors are covered states. But `get_stuck_steps_for_workflow` then returned only those ancestors, never the `catch.Begin` frontier. So every sweep faithfully resumed the ancestors, which re-checked their children, found the catch step still non-terminal, and stayed put — the sweep resumed the wrong steps forever while the real blocker sat untouched. The step recovered only if some *other* path (a parent re-evaluation happening to mark it dirty and generate a continuation) reached it — indirect, incidental, and paced far slower than the fan-in produced new catch frontiers. Small runs had few enough frontiers that the incidental path eventually cleared them; large runs produced them faster than they cleared.

We name this **coverage drift**: a self-healing recovery loop carries a set of states it rescues, the system carries a growing set of states at which work can strand, and nothing enforces that the first covers the second. Each new blocking state in the machine is a silent opportunity to open a liveness hole, and the hole is invisible to tests (the recovery loop is rarely unit-tested against every state) and to code review (the two sets live in different files from the state machine that should govern them).

7. The fix and its verification

The fix is deliberately the *opposite* of the reverted one in risk profile: no write semantics, no new task creation, no feedback path. We defined the stall-state set **once**, canonically, next to the state machine it derives from:

```
STUCK_STEP_STATES = ( EVENT_TRANSMIT, ...block states...,
                      CATCH_BEGIN, CATCH_CONTINUE )
```

and referenced it from every sweep site in both persistence backends. The sweep now fetches and resumes catch-stranded steps directly, via its existing, idempotent resume path. Defining the set in one place is the structural remedy for coverage drift: adding a blocking state to the machine now has one obvious place to register it, and the two persistence implementations can no longer disagree (a divergence that, in this codebase, has independently caused other coordination bugs). Parity tests assert both backends surface catch-stranded steps identically.

Verification was the workload that had never once completed without babysitting: the full 25-region atlas (catch exercised by a failed region), run with the reseed driver *removed*. It completed **hands-off in 18 minutes** — runner completed, zero reseed-driver tasks, zero dead-letters, rankings byte-identical to every correct prior run. The progress trace shows the mechanism working: the tail plateaued at three non-terminal steps for about five minutes, then one for about five more, then finished — each plateau a catch frontier waiting for the next jittered sweep cycle to resume it, `active=0` throughout confirming the recovery came from the *sweep*, not from re-seeding. Against the prior baseline (six to twenty manual reseed generations plus operator endgame nudges, every deep run), the difference is categorical: from never-unattended to unattended.

The residual is honest and small: a catch frontier waits up to one sweep cycle (~5 min) before recovery — latency, not a stall. Eliminating it turned out to be non-trivial, which is itself instructive. The obvious follow-up — seed a continuation for the catch step at catch-entry, so it never depends on the sweep — was attempted twice and reverted twice, each attempt landing on a different property of the same iteration model. Seeding it where updated steps are enumerated failed because the continuation generator, on the path that actually runs, receives a reset change set and derives targets only from a dirty-block set. Marking the catch step's own id into that dirty set failed because the iteration boundary subtracts already-processed steps from its target set, and the step was just processed *into* the catch state. The genuine fix must carry self-reprocess states across the processed-steps subtraction — a change to the core loop whose exact processed/dirty/push interplay produced the catch defects of the sibling parity-gap paper. We deferred it: the payoff is tail latency on an already-correct, hands-off system, and the blast radius is the most delicate loop in the engine. The episode reprises the paper's own lesson at smaller scale — the iteration model, like the coordination substrate, does not reveal its behavior to local reading; both times the fix "obviously" belonged somewhere it did nothing, and only running the system showed it.

8. Related work and discussion

Self-healing and safety nets. Reconciliation loops (Kubernetes controllers), reapers, and watchdogs all encode a target set of unhealthy states to act on. The coverage-drift failure mode is latent in all of them: the target set is authored once and the state space grows around it. Our contribution is to name the mode and prescribe the structural remedy — a single authoritative set co-located with the state machine — rather than the usual per-incident patch that re-opens on the next new state.

Liveness testing. Liveness (something good eventually happens) is notoriously harder to test than safety (nothing bad happens); a missing recovery transition produces no error, no crash, no failed assertion — only the absence of progress, which no unit test naturally waits for. Model checkers (TLA+) can express and check liveness under fairness, but a spec of this engine would model the sweep as "eventually resumes any stuck step" — abstracting away the very hardcoded state set whose incompleteness was the bug. As with the sibling parity-gap finding, the defect lived below the altitude at which a reasonable model operates.

Reasoning vs. observation. The disproven first fix is the paper's methodological core. It was not careless: it was reasoned from real evidence (the CAS conflicts) to a plausible mechanism, and its author (a human–AI pair, per the thesis's authorship addendum) was confident. It failed because concurrent-liveness mechanisms are not locally legible in code — the fault is in interleavings and in the *absence* of an action, neither visible at a call site. The read-only probe succeeded because it read the *effect* (a step frozen in a specific state with nothing pointing at it) rather than inferring the *cause*, and the single-continuation experiment confirmed the mechanism by perturbation-of-one. The rule we draw — observe the frozen system before mutating a write path — is not a counsel of caution but of *evidence order*: in a concurrent system the state is more truthful than the source.

9. Threats to validity and scope

The fix is verified for the **catch-frontier** variant of the stall, which is the variant the current atlas exhibits and reproduces on demand. Earlier, pre-catch stalls in this workload's history were described in terms of block-level "Begin" states; whether those had an additional or distinct mechanism (a sweep per-invocation bound, a lease-hold interaction) is not established here — the current runtime's reproducible stall is the catch-coverage gap, and we scope the claim to it. We report one workload on one four-host fleet; the hands-off result is a single (repeatable) observation, not a distribution. The ~5-minute residual latency means the fix restores *correctness and unattendedness*, not *speed* — a system needing low fan-in-tail latency would want the catch-entry continuation follow-up. Finally, coverage drift is argued as a general mode from one instance; we claim the mechanism and remedy transfer, not a frequency across systems.

10. Conclusion

A workflow froze at its tail for months, and the safety net built to prevent exactly that watched it freeze — because the net's list of states-to-rescue had never grown to include the states error-recovery introduced. The first fix, reasoned confidently from a real symptom, changed write semantics and turned the freeze into a storm; the second, forbidden from touching write paths until it had read the frozen system, found a single stuck step, proved it processable in six seconds, and closed the hole with one canonical list. The workload that had never finished unattended now does. The bug was small; the two lessons are not — *coverage drift* is a liveness hole latent in every self-healing loop, and *observe before you mutate* is the evidence order that concurrent-liveness debugging demands.

Artifacts: disproven fix `a4c8ad4` (reverted `b13b927`); the read-only probe (`docs/thesis/artifacts/stall_probe.py`) and the single-continuation confirmation; the fix `077a264` (canonical `STUCK_STEP_STATES` in `states.py`, both persistence backends, parity tests `tests/runtime/test_liveness_sweep.py`); engineering doctrine `docs/architecture/lessons-learned.md` §25; the hands-off verification run of `osm.emergency.flows.ContinentalEmergencyAtlas` on fleet v96 and its persisted records.